lack of success. Implicitly, you're also testing that the database failure doesn't trigger another error.

The feature spec has the same basic structure, but actually fills out a form. I should say here that you need to be careful about using test doubles with integration tests. The whole point of integration tests is that they cover how objects fit together, so using test doubles potentially weakens the tests. There are cases, though, where an error condition is just easier to simulate with a test double than with user activity. For example, this test simulates a failure of the CreatesProject workflow:

mocks/01/spec/system/add_project_spec.rb

```
it "behaves correctly in the face of a surprising database failure" do
  workflow = instance_spy(CreatesProject,
    success?: false, project: Project.new)
  allow(CreatesProject).to receive(:new)
    .with(name: "Real Name",
          task_string: "Choose Fabric:3\r\nMake it Work:5")
    .and_return(workflow)
  visit new_project_path
  fill_in "Name", with: "Real Name"
  fill_in "Tasks", with: "Choose Fabric:3\nMake it Work:5"
  click_on("Create Project")
  expect(page).to have_selector(".new_project")
end
```

You do the same dance here, setting up a spy to sit in for the workflow object and making sure that the new method returns it. The spy sets success? to fail, and specifies a return value project, since for Rails-internal reasons, the form on the "new" page needs an actual ActiveRecord. You also use the with modifier to make sure the controller is sending the right values to the workflow. If you specify allow and with on a method, and you send arguments that don't match the with arguments and the test fails, you don't fall through to the original method.

As you find other error conditions, try to specify them in the workflow-level test, because workflow errors should not leak out of the workflow object other than to have the workflow object report that it was not successful.

Using Mocks to Specify Behavior

In addition to merely replacing expensive method calls, test doubles enable a different style of testing where you validate the application's behavior rather than its ending state. In most of the tests you've seen throughout the book, the test validates the result of a computation: it's testing whether something is true at the end of an action. When using doubles, however, you have the